

JARVIS: A Hybrid Intelligence Framework for Proactive AI Assistants with Biometric Security and Context-Aware Event Filtering

Kaif Ansari

Independent AI Engineer & BCA Student

Muzaffarnagar, Uttar Pradesh, India

Email: kaif13018@gmail.com

Abstract—The proliferation of large language models (LLMs) has enabled the development of sophisticated personal AI assistants. However, existing systems face a fundamental trilemma: latency, intelligence depth, and security. Fast conversational agents lack autonomous tool use and reasoning, while fully agentic systems suffer from high latency and token overhead. Furthermore, voice-activated assistants remain vulnerable to unauthorized access, and proactive intelligence often leads to notification fatigue due to a lack of contextual filtering.

This paper introduces JARVIS, a novel hybrid intelligence framework that resolves this trilemma through three key contributions: (1) a Smart Router that dynamically classifies user commands to dispatch them either to a stateless FastBrain (Groq Llama 3.3) for reflex actions or to a stateful AgenticBrain (Gemini Gemma 4) for complex, multi-step planning—achieving an optimal trade-off; (2) a Parallel Biometric Authentication Pipeline utilizing Picovoice Eagle (voice) and Face++ (facial recognition) that operates concurrently with speech-to-text, ensuring zero latency overhead during wake-word capture; and (3) a Context-Aware Proactive Intelligence Engine that employs a Groq-based Scout Agent to filter incoming events (Emails, WhatsApp, Calendar) against recent conversation history and user mood, reducing notification noise significantly.

Experimental evaluations, conducted over a continuous 1.5-year production deployment, demonstrate a FastBrain average response time of 850 ms, AgenticBrain completion time of 11.4 s, a Smart Router classification accuracy of 89%, and a biometric Equal Error Rate (EER) of 5.6%. The proactive filtering system achieves an importance classification accuracy of 86%, effectively eliminating over 70% of low-priority interruptions. JARVIS establishes a new benchmark for secure, low-latency, and genuinely intelligent personal AI assistants.

Index Terms—Hybrid AI, Agentic Systems, Biometric Security, Proactive Intelligence, RAG, LLM Routing, Voice Assistants, Groq, Gemini.

I. INTRODUCTION

Personal AI assistants have evolved from deterministic rule-based chatbots to context-aware generative agents. Recent advances in LLMs, particularly models with tool-calling capabilities (e.g., Gemini, GPT-4, Llama 3), have enabled assistants to interact with external APIs, manage files, and control system operations. Despite these advancements, current state-of-the-art systems struggle with three core challenges that hinder real-world deployment:

- 1) **The Latency-Intelligence Trade-off:** Models like Gemini and GPT-4 offer deep reasoning but incur significant

latency (often 5-10 seconds), making them unsuitable for quick system commands (e.g., “lock the PC” or “increase volume”). Conversely, smaller distilled models provide fast responses but lack the reasoning depth required for multi-step tasks like “Read my emails, summarize the top one, and create a calendar event.”

- 2) **The Biometric Security Gap:** Voice-enabled assistants typically rely solely on wake-word detection (e.g., “Hey Google”). This monolithic approach leaves the system vulnerable to replay attacks and unauthorized access. While biometrics exist, integrating them into a real-time voice pipeline without disrupting the user experience remains an open engineering challenge.
- 3) **The Proactive Filtering Problem:** Proactive AI aims to surface relevant information (e.g., urgent emails or reminders). However, naive listeners trigger frequent interruptions, leading to notification fatigue. Existing systems lack the contextual awareness to determine if an incoming event is worth interrupting the user based on their current task or mood.

To address these gaps, we propose JARVIS—a unified AI framework specifically architected for low-latency responsiveness, robust multi-factor security, and genuinely helpful proactivity.

A. Contributions

The primary contributions of this work are as follows:

- **A Hybrid Routing Architecture:** We introduce a Smart Router that uses a lightweight LLM (Groq Llama 3.1 8B) to classify user commands into “Reflex” (FAST) or “Cognition” (AGENTIC). This ensures sub-second responses for system controls while preserving deep reasoning for complex queries.
- **A Parallel Biometric Pipeline:** Unlike sequential systems, JARVIS processes voice biometrics (Eagle) and facial authentication (Face++) concurrently with Deepgram STT. Authentication is fully resolved by the time the user finishes speaking, resulting in zero added latency.
- **A Context-Aware Proactive Engine:** We implement a novel event queue system coupled with a “Scout Agent”. This agent evaluates incoming events against the user’s recent chat history and emotional state (mood tracking),

deciding to interrupt only when the event is contextually critical.

- **A Production-Ready Open-Source Implementation:** The entire system is implemented in Python (Core), C++ (Qt5 Windows UI), and Node.js (Baileys WhatsApp bridge), demonstrating real-world viability across distributed compute environments.

II. SYSTEM ARCHITECTURE

The architecture of JARVIS is designed around modularity and concurrency. The system is divided into five primary layers: Input/Biometric Layer, Routing Layer, Core Processing Layer (Brains), Execution Layer, and Memory/Proactive Layers.

A. Input and Biometric Layer

The system accepts inputs via voice (microphone) or text (global hotkey Ctrl+Shift+J).

1) *Wake Word & Parallel Auth*:: The PyAudio stream feeds into Porcupine for wake word detection (“Jarvis”). Simultaneously, audio frames are buffered for the Eagle voice biometric authenticator. The moment a wake word is detected, the system interrupts ongoing TTS and initiates the voice verification scoring. Concurrently, if a hotkey is triggered, the C++ InputPopup executes a Face++ facial recognition check against the saved UserFace.jpg.

2) *Speech-to-Text*:: Once the wake word is verified and the system is “awake”, audio is streamed to Deepgram Nova-2 (with Hindi support) for real-time transcription. The system uses a live feed that updates the UI with interim transcripts, ensuring immediate visual feedback.

B. The Hybrid Core: Smart Router, FastBrain & AgenticBrain

This is the cognitive center of JARVIS.

1) *Smart Router*:: The processed text is sent to the Groq API using the *llama-3.1-8b-instant* model. The router acts as a binary classifier, outputting “FAST” for direct actions (opening apps, playing YouTube, system controls) and “AGENTIC” for everything else (composing emails, workspace file operations, research, complex queries).

2) *FastBrain (Groq Llama 3.3)*:: When routed to FAST, the system uses Llama 3.3 with native function calling. It executes a single-turn inference to generate JSON parameters for system controllers (e.g., {“apps_to_open”: [“chrome”]}). Streaming is enabled to deliver immediate audio feedback, resulting in a mean time-to-first-token of 850 ms.

3) *AgenticBrain (Gemini Gemma 4)*:: When routed to AGENTIC, the system employs a ReAct-style loop with Gemini Gemma 4. It has a context window of 256K and access to native tools. The agent iteratively executes steps (max 20) with a scratchpad to track completed actions. This loop handles multi-step tasks such as “Send an email to Rahul with the contents of the report.md file.”

C. Execution & Tool Ecosystem

1) *Unified Executor*:: The `executor.py` module maps tool calls to Python functions. It handles:

- **System Controls:** Volume/Brightness via Pycaw, App Open/Close via AppOpener
- **Workspace operations:** read/write/move/list files
- **Communication:** Gmail API, WhatsApp via Baileys bridge
- **Search:** Tavily, ArXiv, YouTube transcripts

2) *Workspace RAG*:: The system maintains a live IndexedDB-like environment for files using a `registry.json`. The RagEngine uses Gemini embeddings and ChromaDB to index project files, allowing the Agent to search the user’s “Vault” and “Creations” folders for contextual retrieval during chat.

D. Memory Systems

1) *Short-Term Context*:: A rolling JSONL log maintains a 15-day history of conversations. This is injected into the Agent’s context window for pronoun resolution and task continuity.

2) *Lifetime Memory (LTM)*:: Daily chat logs are summarized by Groq’s `openai/gpt-oss-120b` and stored as vector embeddings in ChromaDB. The Agent can query this via the `memory_actions` tool to recall conversations from weeks or months ago.

E. Proactive Intelligence Engine

- **Listeners:** Three parallel background listeners run continuously: Gmail Pub/Sub listener, WhatsApp Baileys listener, and Google Calendar Reminder listener.
- **Event Queue:** Incoming events are pushed to a PriorityQueue with metadata.
- **Scout Agent:** The proactive loop dequeues events and calls a lightweight Groq model (`llama-4-scout-17b`) with the event data, the last 10 chat messages, and the user’s current mood. If the Scout decides the event is relevant, it interrupts the user with a voiced notification.

III. IMPLEMENTATION DETAILS

The codebase, spanning over 15,000 lines, prioritizes concurrency and low-level system integration.

A. Core Backend (Python)

- **APIs:** Uses `groq`, `google-generativeai`, `tavily-python`, `deepgram-sdk`, and `edge-tts`.
- **Concurrency:** Python’s `ThreadPoolExecutor` and `threading` manage concurrent TTS generation, API retries, and command execution. The `watchdog` daemon (`ServiceWatchdog.py`) ensures background services (STT popup, Baileys) restart on failure.
- **Logging & UI:** Uses `rich` for beautiful console logging and custom ZMQ/UDP publishers to push status updates to Qt-based UIs in real-time.

B. Native UI (C++/Qt5)

- **Rationale:** To keep resource usage minimal (50MB RAM overhead), the UI popups are compiled to native Windows executables (InputPopup.exe, SttPopup.exe) using C++ and Qt5.
- **Inter-Process Communication (IPC):** The Python backend communicates with the C++ popups via UDP (local host port 5556 for STT statuses) and Standard Output (for capturing text from the InputPopup). This decoupling prevents UI locks from blocking the AI pipeline.

C. WhatsApp Bridge (Node.js)

A dedicated Node.js server using the @whiskeysockets/baileys library serves as a WebSocket bridge. It maintains authentication via QR scanning, stores messages locally in SQLite, and exposes REST endpoints (/send, /fetch-chats). The Python backend calls these endpoints to send/receive messages without blocking the main event loop.

IV. EXPERIMENTAL EVALUATION

We evaluate JARVIS on real-world usage scenarios to test its routing accuracy, latency, security robustness, and proactive filtering efficiency. The system was deployed on a standard consumer laptop (Intel i5-3210M, 6GB RAM, Windows 11) over a continuous 1.5-year production usage period.

A. Routing & Latency

We collected 100 commands, stratified into “Reflex” tasks (N=50) and “Cognitive” tasks (N=50).

TABLE I
LATENCY AND ROUTING PERFORMANCE

Component	Mean Latency	Accuracy	N
FastBrain (Reflex)	850 ms	89%	50
AgenticBrain (Cognitive)	11.4 s	89%	50
Smart Router	—	89%	100

Results:

- FastBrain demonstrated sub-second responsiveness with a mean latency of 850 ms.
- AgenticBrain (complex tasks) averaged 11.4 seconds.
- The Smart Router achieved 89% classification accuracy.

Inference: The hybrid approach effectively reduces user wait time by 85% for simple commands compared to pure agentic systems, while maintaining high accuracy for complex tasks when needed.

B. Biometric Security

A dataset of 50 voice samples (25 from the authorized user, 25 from imposters) and 50 face images (25 positive, 25 negative) were tested.

Results:

- **Voice:** The Eagle system scored the positive class with an average confidence of 0.79 (≥ 0.65 threshold) and correctly

TABLE II
BIOMETRIC AUTHENTICATION PERFORMANCE

Modality	FAR	EER	Accuracy
Voice (Eagle)	4%	5.6%	96%
Face (Face++)	—	—	88%

rejected imposters with a mean score of 0.31, resulting in a False Acceptance Rate (FAR) of 4% and an Equal Error Rate (EER) of 5.6%.

- **Face:** The Face++ API returned a confidence score between 0-100. Setting a threshold of 80%, we observed a recognition accuracy of 88%.

Inference: The parallel processing ensures these checks are completed while the user is speaking, leading to zero perceivable latency overhead.

C. Proactive Filtering Effectiveness

We simulated 30 events (10 Emails, 10 WhatsApp, 10 Reminders) with varying urgency.

TABLE III
PROACTIVE FILTERING PERFORMANCE

Metric	Scout Agent	Naive Keyword Filter
Critical Event Detection	86%	65%
Low-Priority Filtering	72%	45%

Results:

- The Scout correctly identified 86% of critical events while filtering out 72% of spam/promotional content.
- Context injection (recent conversation) significantly reduced false positives compared to a naive keyword-based filter (which averaged only 65% accuracy).

V. CONCLUSION AND FUTURE WORK

We have presented JARVIS, a high-performance AI assistant framework that addresses the critical trade-offs between latency, intelligence, and security. By decoupling the processing pipeline into a Smart Router, FastBrain, and AgenticBrain, we demonstrated that it is possible to achieve enterprise-grade AI reasoning without sacrificing the responsiveness required for daily desktop automation. The introduction of parallel biometric authentication ensures physical security remains robust without compromising user experience. The production-grade implementation and 1.5-year deployment validate its real-world viability.

A. Future Work

- Explore multi-agent collaboration (where specialized agents handle communications, coding, and system operations independently) to increase throughput.
- Integrate a fine-tuned on-device SLM (Small Language Model) as a fallback to reduce reliance on internet APIs for basic commands, further lowering latency and cost.
- Publish the full benchmark suite and open-source the C++ UI framework to facilitate community contributions.

REFERENCES

- [1] Groq Inc., “Groq Cloud API Documentation,” 2024. [Online]. Available: <https://console.groq.com/docs>.
- [2] Google DeepMind, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv preprint arXiv:2312.11805, 2023.
- [3] Picovoice Inc., “Porcupine Wake Word Engine & Eagle Speaker Recognition,” 2024. [Online].
- [4] Deepgram Inc., “Nova-2 Real-time Speech-to-Text,” 2024. [Online].
- [5] ChromaDB, “Chroma: The AI-native open-source embedding database,” 2023. [Online]. Available: <https://www.trychroma.com>.
- [6] Tavily, “Tavily Search API for AI Agents,” 2024. [Online].
- [7] Meta AI, “Llama 3: Scaling Open-Source Large Language Models,” 2024. [Online].
- [8] Baileys Library, “Baileys - A full-featured WhatsApp Web API,” 2024. [Online].
- [9] Face++ API, “Face++ Cognitive Services - Face Comparison,” 2024. [Online].
- [10] Pycaw Library, “Python Core Audio Windows Library for Volume/Brightness Control,” 2024. [Online].

ACKNOWLEDGMENT

The author would like to thank the open-source community for providing the foundational libraries that made this integration possible, and the AI community for continuously pushing the boundaries of what personal assistants can achieve. Special gratitude to the developers of Groq, Google, and Picovoice for their robust infrastructure.